SETUP GUIDE

ExtendedReach Report Sync

Automatically download your ExtendedReach reports every day and file them in Google Drive.

Written for someone who does not write software. Every command is written out in full. Nothing assumes prior knowledge.

What you are setting up

Right now, someone signs in to ExtendedReach, opens a report, clicks **Excel**, and saves the file. Then does it again for the next report. Every day. This tool does that same sequence for as many reports as you configure — nine, or any other number — in one browser session, on a timer, and puts the results in Google Drive so the leadership team always has today's numbers.

It is deliberately boring. It clicks the same buttons a person clicks. It cannot create, edit, approve, reject, submit or delete anything in ExtendedReach — that is enforced in the code in three separate places, not just promised.

> **Two things worth knowing before you start**
> **1.** Setup takes about an hour, once. After that it runs on its own.
> **2.** The reports contain children's names, dates of birth and Medicaid numbers. Everything in this guide is designed around that fact. Nothing shortcuts it.

Contents

Before you begin

Five things to have ready. Gather them now and the rest goes smoothly.

1. A Mac that stays on

The tool runs on your Mac, not in the cloud. macOS will not wake a sleeping Mac to run a scheduled job, so if the laptop is closed at 6 PM, nothing happens that evening. It catches up at the next wake. If this needs to be dependable, use a Mac that stays awake and signed in.

2. Your ExtendedReach login

The username, password and phone or app you use for the two-factor code. You will type these into the real ExtendedReach login page yourself, once. You never put them in a file.

3. The reports you want automated

List them, and know how you reach each one: which menu, which submenu. You will open each while the tool watches. **Start with one**, get it all the way through to Drive, and only then add the rest — a problem is far easier to find with one report than with nine.

4. A Google Drive folder

Create the folder where reports should land. Open it in your browser and look at the address bar. You need the long jumble of characters after `/folders/`:

```
https://drive.google.com/drive/folders/1AbCdEf...XyZ
                                       ^^^^^^^^^^^^^
                                       this part
```

5. About 60 minutes

Most of it is waiting for downloads and clicking through Google's permission screens. The parts that need your attention are Steps 5 and 7. Add roughly five minutes per extra report beyond the first.

How you will type commands

This guide asks you to type things into an app called **Terminal**. It comes with every Mac. To open it: press **Command** + **Space**, type **Terminal**, press **Return**.

A black or white window opens with a blinking cursor. When this guide shows a shaded box like the one below, type it exactly (or copy and paste it) and press **Return**.

```
echo hello
```

The window answers:

```
hello
```

Terminal on its own, or VS Code?

Use VS Code. You will be doing two kinds of work — running commands, and editing two settings files — and VS Code does both in one window, with a Terminal built into the bottom of it.

There is also a specific trap it avoids. TextEdit, the editor that opens by default on a Mac, substitutes curly quotes for plain ones as you type. The settings files need plain quotes. The two are almost impossible to tell apart on screen:

> **"Status"**
> plain quotes. correct.
>
> **“Status”**
> curly quotes. breaks the file.

The tool now spots this and says so in as many words, so it is no longer a mystery if it happens — but not having it happen is better. VS Code never substitutes quotes.

To install it, if you do not have it

Download from `code.visualstudio.com`, open the downloaded file, and drag **Visual Studio Code** into your Applications folder. Then open it, press **Command + Shift + P**, type **shell command**, and choose **Install 'code' command in PATH**. That last step is what lets you type `code` to open a file.

Opening this project in VS Code

```
cd ~/Documents/milli/agents/extendedreach-report-sync-poc
code .
```

The `.` means "this folder". VS Code opens with the project files listed down the left. Press **Control + `** (the backtick key, above Tab) to open a Terminal at the bottom — already in the right folder. Every command in this guide can be typed there.

If you would rather not install anything, the Terminal app alone works for everything here. Where this guide says `code somefile`, use `open -e somefile` instead — and turn off TextEdit's smart quotes first, under **Edit -> Substitutions**.

> **If a command seems to hang**
> Some commands take a minute or two with no visible progress — installing the browser in Step 2 is the slow one. That is normal. You will get your cursor back when it finishes. Do not close the window.

How it works, in plain English

Nine things happen, in order, every time it runs.

1. **Opens a browser**
   A real Chrome window, using a saved profile that remembers you are signed in.
2. **Checks you are still signed in**
   It looks for one specific thing on the page that only appears when signed in. It does not read the page.
3. **If not signed in, it stops**
   On a scheduled run it stops and does nothing. It will never type your password or a two-factor code. Ever.
4. **Goes to your report**
   Straight to the address you gave it in Step 5.
5. **Clicks the export button**
   The same button you click by hand.
6. **Saves the file**
   Into a folder on your Mac, renamed with today's date so files never overwrite each other.
7. **Checks the file is real**
   The important step. See below.
8. **Uploads it to Google Drive**
   Only if step 7 passed. Never before.
9. **Writes down what happened**
   A log you can read later, with all personal details stripped out.

Why step 7 matters more than it sounds

When a website session expires, the site often does not show an error. It quietly serves a login page instead. Your browser saves that login page under the filename it expected — so you end up with a file called something like `report.xlsx` that is not a report at all.

A tool that only checked "did a file arrive?" would upload that happily, every day, and the folder would fill with junk that looks correct in a file listing. This tool opens the file and confirms it is a real spreadsheet with the columns you expect. If it is not, **nothing is uploaded** and the run is recorded as failed.

> **Where your password goes**
> Into the real ExtendedReach login page, typed by you, once. It is never stored in a settings file, never in the code, never in a log, and never sent anywhere. After you sign in the first time, your Mac's browser profile remembers the session — the same way your browser keeps you logged in to a website between visits.

STEP 1 Get the software onto your Mac

5 minutes

The tool lives inside the same project as your dashboard. Open Terminal and go to that folder. If your project is somewhere other than your Documents folder, change the path to match.

```
cd ~/Documents/milli
```

Now fetch this tool and switch to it:

```
git fetch origin
git checkout claude/extendedreach-report-sync-poc-a5jkhn
cd agents/extendedreach-report-sync-poc
```

Check you are in the right place:

```
ls
```

You should see these names listed:

```
README.md config docs requirements.txt scripts src tests
```

> **If 'git checkout' says the branch is not found**
> Run `git fetch origin` on its own first, wait for it to finish, then try the checkout again. If it still fails, send me the exact message.

STEP 2 Install it

10-15 minutes, mostly waiting

One command does everything:

```
./scripts/install_playwright.sh
```

This sets up a private workspace for the tool, downloads what it needs, downloads a copy of the Chrome browser for it to drive, and then runs 99 self-tests. The browser download is the slow part.

When it finishes you will see:

```
99 passed
Setup complete.
Next:
```

Those 99 tests passing means the tool's own logic is sound — the file checking, the naming, the safety rules. It does not yet mean it works against your portal. That is what Step 7 is for.

The one command to remember

If you ever lose your place in this guide, run this. It tells you which steps are done and gives you exactly one command to run next.

```
./.venv/bin/python -m src.main --doctor
```

STEP 3 Create your settings file

10 minutes

The tool reads its settings from a file named `.env`. Make your own copy of the example:

```
cp .env.example .env
```

Then open it for editing:

```
code .env
```

(Or `open -e .env` if you are using TextEdit rather than VS Code.)

You will see a long file with explanatory notes. Lines starting with # are notes — ignore them. You are looking for lines containing the word **TODO**. Replace each one.

What to put where

Setting	What to put
EXTENDEDREACH_BASE_URL	The web address you sign in to, e.g. https://youragency.extendedreach.com
REPORT_SLUG	A short nickname you choose for this report. Lowercase, use underscores instead of spaces. Example: past_due_tasks
BROWSER_PROFILE_DIR DOWNLOAD_DIR LOG_DIR SCREENSHOT_DIR	Four folders on your Mac. Replace TODO_YOU with your Mac username and leave the rest. They are created for you.
GOOGLE_DRIVE_FOLDER_ID	The jumble of characters from your Drive folder address (see page 2).
GOOGLE_CREDENTIALS_FILE GOOGLE_TOKEN_FILE	Replace TODO_YOU with your Mac username. Step 4 puts a file here.
EXPECTED_CSV_HEADERS	Leave the TODO for now. You fill this in at Step 6, once you have seen a real file.

Not sure of your Mac username? Run this and use what it prints:

```
whoami
```

Save the file (**Command** + **S**) and close it.

> **Why the folders must be outside the project**

> Those four folders hold your signed-in browser session and the downloaded reports — real case data. The project folder is tracked by version control and gets shared. The tool refuses to start if you point it at a folder inside the project, so a report can never be published by accident. If you see that error, that is the safety working.

STEP 4

Connect Google Drive

10 minutes

Google needs to know this tool is allowed to put files in your Drive. This part is all clicking in a web browser.

A. Create a Google project

Go to `console.cloud.google.com` and sign in with the Google account that owns your Drive folder. At the top of the page there is a project selector. Create a new project and call it something like **ExtendedReach Sync**.

B. Switch on the Drive connection

In the menu on the left, choose **APIs and Services**, then **Library**. Search for **Google Drive API**. Open it and press **Enable**.

C. Create the credentials

Still under **APIs and Services**, choose **Credentials**. Press **Create credentials** and pick **OAuth client ID**.

If it asks you to configure a consent screen first, do that: choose **Internal** if it is offered, give it a name, and enter your own email in the contact fields. You can skip the optional sections.

Then, for application type, choose **Desktop app**. This matters — the other types will not work. Name it anything. Press **Create**, then **Download JSON**.

D. Put the file where the tool expects it

A file lands in your Downloads folder with a long name starting `client_secret`. Move and rename it:

```
mkdir -p ~/er-sync
mv ~/Downloads/client_secret*.json ~/er-sync/credentials.json
```

Confirm it arrived:

```
ls ~/er-sync
```

Should print:

```
credentials.json
```

> **Treat that file like a password**
> `credentials.json` lets software act on your Google account. Do not email it, do not put it in a shared folder, do not add it to the project folder. It stays in `~/er-sync` on your Mac.

STEP 5 Show it your reports

10 min for the first, 3 each after

The tool does not know where your reports live or what the export button looks like. This step teaches it, by watching you. You run it **once per report** — nine times for nine reports.

```
./.venv/bin/python -m src.main --setup-assist
```

It asks for a short name for the report first (your choice — lowercase, underscores instead of spaces, e.g. `past_due_tasks`). Then:

A browser window opens and the Terminal walks you through three parts.

Part 1 — Sign in

Sign in to ExtendedReach in that window, exactly as you normally would, including the two-factor code. Take as long as you need. Then click back to the Terminal window and press **Return**.

It shows you a numbered list of things it found that only appear when signed in — usually a **Sign Out** link. Type the number next to the most obvious one and press **Return**.

Part 2 — Open the report

In the browser, navigate to the report you chose, the same way you always do. If it has filters you want applied every time — a date range, a program — set them now. Then press **Return** in the Terminal.

It prints the address it captured. Check it looks right.

Part 3 — Point at the export button

Do not click the export button. Just look at the numbered list in the Terminal and pick the one that matches it — often labelled **Excel**. Type its number and press **Return**.

Then save what it captured:

```
cp config/workflow.draft.json config/workflow.json
```

Now repeat for each remaining report

Run the same command again. It **adds** the next report and keeps the ones you have already done — it never starts over. After the first time it will not ask you to sign in again either, so each extra report takes two or three minutes.

```
./.venv/bin/python -m src.main --setup-assist
cp config/workflow.draft.json config/workflow.json
```

Check what you have so far at any point:

```
./.venv/bin/python -m src.main --list-reports
```

```
9 report(s) configured
    slug           columns checked  description
[x] past_due_tasks 4                Past due case tasks
[x] open_beds      2                Open beds
...
9 enabled; these all run on each scheduled run.
```

And check everything so far makes sense:

```
./.venv/bin/python -m src.main --validate-config
```

It either says the configuration is complete, or lists exactly what is still wrong. Warnings about `expected_csv_headers` are expected at this point — Step 6 fills those in, and they do not block a run.

> **If it cannot find the export button**
> Some portals label it unusually and the automatic scan misses it. The Terminal will say so and tell you how to find it in Chrome instead: right-click the button, choose **Inspect**, then right-click the highlighted line and choose **Copy** then **Copy selector**. Send me what you copied and I will put it in the right place.

STEP 6 Tell it what a good file looks like

5 minutes

This is how the tool spots a fake file. You give it the column names your report actually has; if a download does not have them, it is rejected and never uploaded.

A. Download each report by hand, once

In your normal browser, open a report and click the export button. Open the file that downloads. Repeat for each report — you only ever do this once per report.

B. Copy the column headings

Look at the very first row — the headings. Write them out separated by commas, exactly as they appear, including spaces and symbols:

```
Case #,Client Name,Task,Due Date,Status
```

C. Put them in the workflow file

Each report has its own columns, so each report gets its own list. Open the workflow file:

```
code config/workflow.json
```

Find your report by its short name, and fill in the `expected_csv_headers` list inside its `validation` section. It looks like this:

```
"past_due_tasks": {
  "enabled": true,
  "validation": {
    "expected_csv_headers": ["Case #", "Task", "Due Date", "Status"]
  }
}
```

Each name in double quotes, commas between them. Do this for every report. Leave the list empty — `[]` — for any report you do not want column-checked; it is still checked for size and file type.

Now confirm:

```
./.venv/bin/python -m src.main --validate-config
```

You want to see:

```
Configuration is complete and internally consistent.
```

> **You do not need every column**

> List the ones that should always be there. Extra columns in the file are fine — reports gain columns over time and that should not break anything. Missing ones are what get caught. Pick four or five distinctive headings rather than all forty.

A useful check that touches nothing

This makes up fake files and proves the checking works, without going near ExtendedReach or Drive:

```
./.venv/bin/python -m src.main --test-download-fixture
```

Six samples are tested. Four should pass. **Two should fail** — a cut-off file and a login page disguised as a spreadsheet. Those two failures are the point: they are what protects your Drive folder.

STEP 7 Test run — nothing gets uploaded

10 minutes

This is the real test: the first time the tool meets your actual portal. It downloads and checks a report but **stops before uploading**, so there is nothing to undo.

```
./scripts/run_once.sh --dry-run
```

A browser window opens. Because you signed in during Step 5, it should already know you. If it asks you to sign in again, do so — it waits.

Watch it navigate, click export, and save the file. Then:

It works through every report in one browser session, then prints a line per report:

```
  past_due_tasks  success
  open_beds       success
  next_court      success
  ...
  9 uploaded, 0 already there, 0 failed of 9 report(s)
```

If one report fails, the others still run. That is deliberate — a single mislabelled button should not cost you the other eight.

Now open the files yourself and look at them

This is the part worth doing carefully.

```
open ~/er-sync/downloads
```

Check three things:

Check each one. This is tedious with nine reports and it is the only time you have to do it.

Check	Why
Is it the right report?	Confirms it went to the right page, not a neighbouring one. With several reports, the failure to watch for is two files that are actually the same export.
Are the filters applied?	If you set a date range in Step 5, confirm the file reflects it.
Does the row count look sane?	A report with 3 rows when you expect 900 means a filter is wrong.

> **If this step fails, that is normal and useful**

> This is the first contact between the tool and your portal. Common outcomes: the export button was mislabelled, a column name does not match, or the page layout differs from what was captured. The tool tells you which, in plain words, and stops safely. Send me the message and the run id and I will fix it — this is the part I could not test in advance because I have never seen your portal.

STEP 8 The first real run

5 minutes

Same as the test, but it uploads. Only do this once you are happy with the file you opened in Step 7.

```
./scripts/run_once.sh
```

The first time only, a browser tab opens asking you to let the tool use Google Drive. Sign in with the account that owns the folder and approve it.

> **If Google warns the app is not verified**
> You will likely see a screen saying the app is not verified. That is expected — you built it, and Google only verifies apps that are published publicly. Choose **Advanced**, then **Go to ... (unsafe)**. It is your own project, approved by you, and access is limited to files this tool creates.

You should end with a Drive file id:

```
status success
file extendedreach_past_due_tasks_2026-08-26_183012.csv
drive id 1XyZ...
```

Open your Drive folder in a browser. The file should be there.

Run it twice on purpose

Run the exact same command again:

```
./scripts/run_once.sh
```

This time it should say:

```
status skipped
drive id 1XyZ...
```

It recognised that today's report is already in the folder and declined to upload a second copy. This is why the schedule can safely run six times a day without cluttering the folder — and why a manual re-run after a problem will not create duplicates.

STEP 9 Turn on the daily schedule

5 minutes

Pick one of these two.

Option A — once a day at 6 PM

```
./scripts/install_schedule.sh daily
```

Simple. One attempt each evening.

Option B — every two hours, weekdays (recommended)

```
./scripts/install_schedule.sh business-hours
```

Tries at 8, 10, 12, 2, 4 and 6, Monday to Friday. Because of the duplicate check you saw in Step 8, you still get **one file per day** — the other five runs notice it is already there and stop. The benefit is six chances to catch a moment when your Mac is awake and the session is valid, instead of one.

Confirm it is registered:

```
launchctl list | grep extendedreach
```

Test it without waiting for the clock:

```
launchctl start com.agency.extendedreach-report-sync
```

Wait a minute, then check what happened:

```
./.venv/bin/python -m src.main --status
```

To turn it off again, at any time

```
./scripts/install_schedule.sh --uninstall
```

This only stops the timer. Your downloads, logs and Drive files are untouched, and you can still run it by hand.

> **It refuses to schedule something that has never worked**
> If you skipped ahead and Step 8 never succeeded, this command will stop and tell you so. A timer on something broken just produces a failure every evening. Finish Step 8 first.

Your weekly two-minute check

Automation fails quietly. This is the habit that catches it.

```
cd ~/Documents/milli/agents/extendedreach-report-sync-poc
./.venv/bin/python -m src.main --status
```

What you might see

All good

```
report          when         status     detail
past_due_tasks  08-27 18:00  success    1XyZ...
open_beds       08-27 18:00  success    1AbC...
...
All 9 report(s) are up to date.
```

Nothing to do.

One report broken, the rest fine

```
ACTION NEEDED: 1 report(s) failing on the last run:
    open_beds     portal_structure_changed
The other 8 are fine, so this is that report's own problem,
not the session.
```

That last line is the one that matters. If one report fails and the others succeed, the sign-in is fine and something changed on that one report's page — send me the report name and the category. If **every** report fails at once, it is almost always the session, not nine broken pages.

Needs you — the most common one

```
ACTION NEEDED: the portal session has expired.
One headed run signs it back in:  ./scripts/run_once.sh
Until then every report is stopped, uploading nothing.
```

Your ExtendedReach session lapsed. This is expected every so often — how often depends on your agency's settings. The tool correctly refused to do anything about it, because the alternative would be automating your login. Run the command it names, sign in, and the schedule picks up again on its own.

Something broke

```
2026-08-26 18:00:00 failed portal_structure_changed
```

Usually means ExtendedReach changed its page layout. Send me the category shown (the part after 'failed') and I will adjust it.

> **Why not just look at the Drive folder?**
> Because an empty folder cannot tell you the difference between "nothing needed uploading" and "it has been failing for six days". The status command can. Two minutes on a Monday is the whole maintenance burden.

Lost your place at any point?

```
./.venv/bin/python -m src.main --doctor
```

Shows the nine setup steps, ticks off what is done, and gives you one command to run next.

When something goes wrong

Every run ends with a number. Zero means fine. Anything else has a specific meaning.

Code	Meaning	What to do
0	Worked, or correctly skipped a duplicate	Nothing
1	Something unexpected	Run --status and send me what it says
2	Settings incomplete	Run --validate-config; it lists what is missing
3	Someone must sign in	Run ./scripts/run_once.sh and sign in. Most common one.
4	The file failed its checks	Often the session expired mid-run and a login page was saved. Nothing was uploaded, which is correct.
5	Could not reach or export the report	Usually the portal layout changed. Send me the category.
6	Google Drive refused the upload	Check the folder id, and that the Google account is right
7	Another run was already going	Nothing. The safety worked.

Specific messages

"resolves inside the git repository"

One of your four folder settings points inside the project folder. Move it to `~/er-sync/...` instead. This guard exists so a downloaded report can never be published by accident.

"chromium did not start"

Re-run the installer:

```
./scripts/install_playwright.sh
```

"is not valid JSON"

Something in `config/workflow.json` is malformed. The message names the line and, where it can, the actual cause — most often curly quotes substituted by a text editor, which look identical to plain ones on screen. Fix what it names, or re-open the file in VS Code, which does not substitute them.

"csv_headers_missing"

The file arrived but a column you listed in Step 6 was not in it. Either the report changed, or a heading was typed slightly wrong. Download the report by hand and compare the first row against your `EXPECTED_CSV_HEADERS` line.

"drive_upload_failed"

If this mentions a 403, the Drive folder was not created by this tool and Google will not let it write there under the limited permission it asks for. Easiest fix: let the tool create its own subfolder, and use that folder's id instead. Tell me and I will set it up.

The schedule runs but nothing appears

Check, in this order:

Check	How
Was the Mac awake at that hour?	macOS will not wake a sleeping Mac for a scheduled job.
Is the job registered?	`launchctl list | grep extendedreach`
What do the runs say?	`./.venv/bin/python -m src.main --status`
Did it ever work by hand?	`./scripts/run_once.sh`

What this tool will never do

These are enforced in the code, not left to good intentions. Worth knowing if leadership asks.

It will never...	How that is guaranteed
Change anything in ExtendedReach	It can only perform actions from a fixed read-only list, and it refuses to visit any address containing words like delete, submit or approve.
Type your password or two-factor code	There is no place in the settings to put them. A check confirms no such field exists anywhere in the code.
Type a person's name into a search box	Filter values must match a strict pattern that excludes free text, apostrophes and commas — which rules out names.
Upload a file it has not checked	There is no path through the code from download to upload that skips validation.
Put case details in a log	Logs are scrubbed automatically, and errors are recorded as fixed category names rather than messages, because error messages from a browser often quote the page.
Screenshot a report page	Screenshots are off unless you turn them on, and even then they are only taken on a login or error page — never one that could show a child's record.
Send a screenshot to Drive	Only the validated report file is ever uploaded.

Two things to keep an eye on

The Google account. Files land in the Drive you chose. Make sure that account is the right one for records of this kind, and that the folder is shared only with people who should see them.

The four folders on your Mac. `~/er-sync/downloads` accumulates real reports over time. They are as sensitive as anything in ExtendedReach. Keep the Mac locked, encrypted (FileVault) and backed up somewhere appropriate — or clear the folder periodically once files are safely in Drive.

Words this guide uses

Word	What it means here
Terminal	The Mac app where you type commands. Command + Space, type Terminal, press Return.
Command	A line you type into Terminal and run by pressing Return.
Repository (repo)	The project folder, with a history of every change. Yours is called milli.
Branch	A named version of the project. This tool lives on a branch so it does not disturb the dashboard.
.env file	Your private settings. Never shared, never uploaded. The dot at the start makes it hidden in Finder.
Selector	A precise way of pointing at one thing on a web page — like a seat number in a theatre. Step 5 captures these for you.
MFA / two-factor	The extra code from your phone or an app when you sign in. This tool never handles it; you always type it yourself.
CSV / XLSX	Two spreadsheet file types. CSV is plain text; XLSX is Excel. Some reports come as one, some the other.
Dry run	A rehearsal. Everything happens except the upload.
Run key	An invisible label of report plus date, attached to each upload. It is how the tool recognises today's file is already there.
Exit code	The number a run ends with. 0 is good; see page 13.
launchd	The part of macOS that runs things on a schedule. Step 9 sets it up for you.
Headed / headless	Headed means you can see the browser window. Headless means it runs invisibly, which is how the schedule runs it.

One-page command reference

Every command starts by going to the project folder. Adjust the path if yours differs.

```
cd ~/Documents/milli/agents/extendedreach-report-sync-poc
```

Day to day

Command	What it does
`./.venv/bin/python -m src.main --status`	Latest run of every report. Says if anything needs you. **Run weekly.**
`./.venv/bin/python -m src.main --list-reports`	Every report configured, and whether it is switched on
`./.venv/bin/python -m src.main --doctor`	Where am I in setup, and what is the one next command
`./scripts/run_once.sh`	Run every enabled report now, for real
`./scripts/run_once.sh --report open_beds`	Run just one report
`./scripts/run_once.sh --dry-run`	Run it now, but do not upload

Setup

Command	What it does
`./scripts/install_playwright.sh`	Install everything (Step 2)
`cp .env.example .env`	Create your settings file (Step 3)
code .env	Edit your settings (open -e .env without VS Code)
code .	Open the whole project in VS Code
`./.venv/bin/python -m src.main --setup-assist`	Add ONE report. Run again for each (Step 5)
`./.venv/bin/python -m src.main --validate-config`	Check the settings make sense
`./.venv/bin/python -m src.main --test-download-fixture`	Prove the file checking works, offline

The schedule

Command	What it does
`./scripts/install_schedule.sh daily`	Run every day at 6 PM
`./scripts/install_schedule.sh business-hours`	Every two hours, weekdays (recommended)
`./scripts/install_schedule.sh --uninstall`	Stop the schedule
`launchctl list | grep extendedreach`	Is the schedule registered?
`launchctl start com.agency.extendedreach-report-sync`	Run the scheduled job right now

> **If you only remember one command**
> **`./.venv/bin/python -m src.main --doctor`**
> It always tells you where you are and what to do next.

This guide is generated from the project itself and regenerated when the tool changes. If a command here does not match what you see on screen, trust the screen and tell me.